

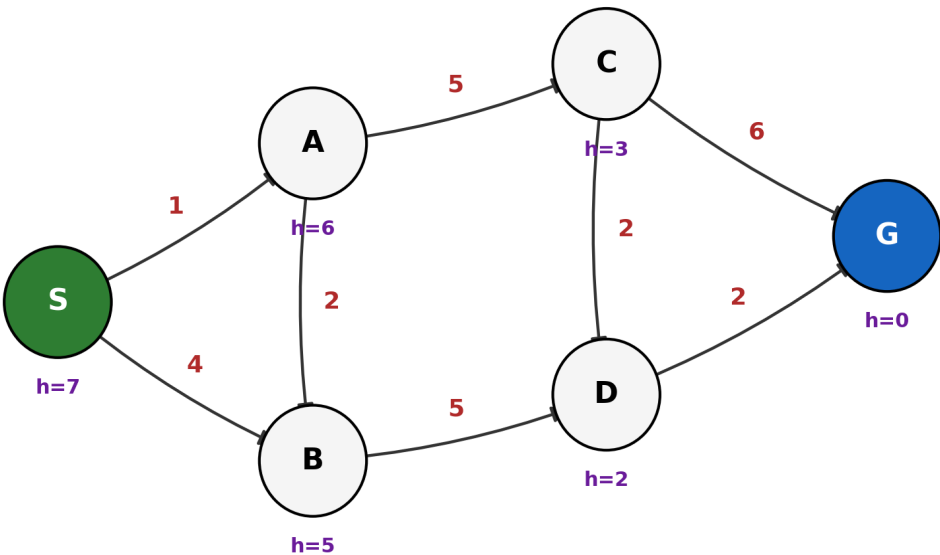
Search & Game-Playing Algorithms

DFS · BFS · UCS · Greedy Best-First · A* · MiniMax · Alpha-Beta Pruning

This document contains two example graphs and a full, step-by-step solution for each of the seven algorithms. **Graph 1** (a weighted state-space graph with heuristic values) is used for DFS, BFS, UCS, Greedy Best-First Search, and A*. **Graph 2** (a two-player game tree) is used for MiniMax and Alpha-Beta Pruning.

Graph 1 — Weighted Search Graph

Graph 1 — Search Graph (edge weights = cost, h = heuristic to G)



Start node = **S**, Goal node = **G**. Numbers on edges are step costs. The purple h-values are straight-line heuristic estimates to the goal (used by Greedy and A*). Children are explored in alphabetical order.

Node	Neighbors (cost)
S	A (1), B (4)
A	B (2), C (5)
B	D (5)
C	D (2), G (6)
D	G (2)
G	— (goal)

Node	S	A	B	C	D	G
$h(n)$	7	6	5	3	2	0

1. Depth-First Search (DFS)

DFS goes as deep as possible down one branch before backtracking. It uses a Stack (LIFO). It does not consider cost or heuristic — only the order of neighbors.

Trace

- Visit S → push neighbors A, B. Go to first: A
- At A → push neighbors B, C. Go to first: B
- At B → push neighbor D. Go to: D
- At D → push neighbor G. Go to: G
- G is the goal → STOP

Path found: $S \rightarrow A \rightarrow B \rightarrow D \rightarrow G$

Total cost: $1 + 2 + 5 + 2 = 10$ (DFS does not guarantee the optimal path in general — here it happens to match the optimal cost).

2. Breadth-First Search (BFS)

BFS explores level by level using a Queue (FIFO). It guarantees the path with the fewest edges, but NOT the lowest total cost (since edges are weighted).

Trace

Step	Queue (before)	Dequeue	Add to queue
1	[S]	S	A, B
2	[A, B]	A	C (B already queued)
3	[B, C]	B	D
4	[C, D]	C	G (D already queued)
5	[D, G]	D	— (G already queued)
6	[G]	G	GOAL REACHED

Backtracking parent pointers from G: $G \leftarrow C \leftarrow A \leftarrow S$

Path found: $S \rightarrow A \rightarrow C \rightarrow G$

Total cost: $1 + 5 + 6 = 12$ (fewest edges = 3, but not the cheapest path)

3. Uniform Cost Search (UCS)

UCS always expands the frontier node with the lowest cumulative path cost $g(n)$ (a Priority Queue). It ignores heuristics and guarantees the optimal-cost path.

Trace

Expand	$g(n)$	Frontier updates
S	0	A=1, B=4
A	1	B updated 4→3 (via A), C=6 (via A)
B	3	D = 3+5 = 8 (via B)
C	6	D via C = 6+2 = 8 (tie, keep via B), G = 6+6 = 12 (via C)
D	8	G updated 12→10 (via D)
G	10	GOAL — expand and stop

Path found: $S \rightarrow A \rightarrow B \rightarrow D \rightarrow G$

Total cost (optimal): $1 + 2 + 5 + 2 = 10$

4. Greedy Best-First Search

Greedy always expands the frontier node with the smallest heuristic value $h(n)$, ignoring the cost already paid $g(n)$. It is fast but NOT guaranteed to be optimal.

Trace

Expand	$h(n)$	Frontier (node : h)
S	7	A:6 , B:5
B (smallest h)	5	A:6 , D:2
D (smallest h)	2	A:6 , G:0
G (smallest h)	0	GOAL — stop

Path found: $S \rightarrow B \rightarrow D \rightarrow G$

Total cost: $4 + 5 + 2 = 11$ — **NOT optimal** (the true optimum is 10, found by UCS/A*). This demonstrates that Greedy can be misled by the heuristic and ignore actual path cost.

5. A* Search

A* expands the node with the smallest $f(n) = g(n) + h(n)$, combining actual cost so far with the heuristic estimate to the goal. With an admissible heuristic, A* is optimal.

Trace

Expand	g	h	f = g+h	Frontier updates
S	0	7	7	A: g=1,f=7 B: g=4,f=9
A (f=7)	1	6	7	B updated: g=3,f=8 C: g=6,f=9
B (f=8)	3	5	8	D: g=8,f=10
C (f=9)	6	3	9	D via C: g=8 (tie, no change) G: g=12,f=12
D (f=10)	8	2	10	G updated: g=10,f=10
G (f=10)	10	0	10	GOAL — stop

Path found: $S \rightarrow A \rightarrow B \rightarrow D \rightarrow G$

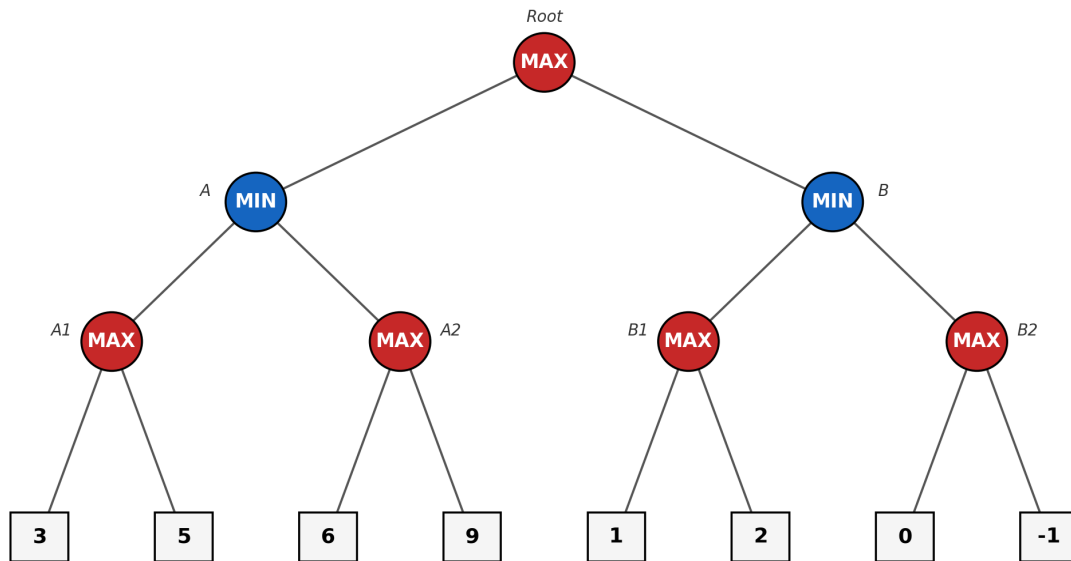
Total cost (optimal): $1 + 2 + 5 + 2 = 10$ — matches UCS, confirming optimality, but A* reached it while expanding fewer/better-guided nodes thanks to the heuristic.

Summary — Graph 1

Algorithm	Path	Total Cost	Optimal?
DFS	S-A-B-D-G	10	Yes (here, not guaranteed)
BFS	S-A-C-G	12	No (fewest edges only)
UCS	S-A-B-D-G	10	Yes
Greedy	S-B-D-G	11	No
A*	S-A-B-D-G	10	Yes

Graph 2 — Game Tree (MiniMax / Alpha-Beta)

Graph 2 — Game Tree (MiniMax / Alpha-Beta Pruning)



Root and level-2 nodes are **MAX** (red), level-1 nodes are **MIN** (blue). Leaf values (payoffs) are shown left to right: 3, 5, 6, 9, 1, 2, 0, -1. Children are always evaluated left to right.

6. MiniMax Algorithm

MiniMax explores the full tree bottom-up: MAX nodes take the maximum of their children, MIN nodes take the minimum, until the root's value is determined.

Bottom-up evaluation

- $A1 \text{ (MAX)} = \max(3, 5) = 5$
- $A2 \text{ (MAX)} = \max(6, 9) = 9$
- $A \text{ (MIN)} = \min(A1, A2) = \min(5, 9) = 5$
- $B1 \text{ (MAX)} = \max(1, 2) = 2$
- $B2 \text{ (MAX)} = \max(0, -1) = 0$
- $B \text{ (MIN)} = \min(B1, B2) = \min(2, 0) = 0$
- $\text{Root (MAX)} = \max(A, B) = \max(5, 0) = 5$

MiniMax value of the game = 5, obtained via the path $\text{Root} \rightarrow A \rightarrow A1 \rightarrow \text{leaf (5)}$. This means the MAX player should choose branch A.

7. Alpha-Beta Pruning

Alpha-Beta Pruning computes the exact same result as MiniMax but skips branches that cannot influence the final decision, using α (best value MAX can guarantee so far) and β (best value MIN can guarantee so far). A branch is pruned whenever $\alpha \geq \beta$.

Trace (α, β passed down; * = pruned)

Node visited	α (in)	β (in)	Result / Action
Root	$-\infty$	$+\infty$	go to A
A1	$-\infty$	$+\infty$	sees 3 $\rightarrow \alpha=3$; sees 5 $\rightarrow \alpha=5$; returns 5
A (after A1)	$-\infty$	5	$\beta = 5$
A2	$-\infty$	5	sees 6 $\rightarrow \alpha=6 \geq \beta=5 \rightarrow$ PRUNE leaf "9" *
A	$-\infty$	5	$\beta = \min(5,6) = 5 \rightarrow$ returns 5
Root (after A)	5	$+\infty$	$\alpha = 5$
B1	5	$+\infty$	sees 1 (no change); sees 2 $\rightarrow$ returns 2
B (after B1)	5	2	$\beta = 2 \leq \alpha = 5 \rightarrow$ PRUNE entire B2 subtree * (leaves 0, -1 skipped)
B	5	2	returns 2 (best found)
Root	5	$+\infty$	$\max(5, 2) = 5 \rightarrow$ FINAL VALUE

Result: 5 (identical to MiniMax), but only 5 of the 8 leaves were actually visited (3, 5, 6, 1, 2) — leaves 9, 0, and -1 were pruned because they could not change the final decision. Best move for MAX = branch A.

Summary — Graph 2

Algorithm	Final Value	Nodes/Leaves Evaluated
MiniMax	5	All 8 leaves
Alpha-Beta Pruning	5	5 leaves (3 pruned: 9, 0, -1)